

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

К защите допустить:

Заведующий кафедрой ИИТ

_____ Д. В. Шункевич

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
по дисциплине: "Проектирование баз знаний"
на тему:

**РЕАЛИЗАЦИЯ ИНТЕРПРЕТАТОРА SC-МОДЕЛЕЙ
ПОЛЬЗОВАТЕЛЬСКИХ ИНТЕРФЕЙСОВ**

БГУИР КП5 1-40 03 01 01 063 ПЗ

Студент гр. 121703
Руководитель

А. М. Рутковский
М. Е. Садовский

Минск 2023

СОДЕРЖАНИЕ

Перечень условных обозначений	3
Введение	4
1 Анализ подходов к созданию веб-приложений	6
1.1 Анализ требований к разрабатываемой системе	6
1.2 Анализ аналогов разрабатываемой системы	7
1.3 Вывод	9
2 Проектирование разрабатываемой системы	11
2.1 Постановка задачи	11
2.2 Потенциальные пользователи разрабатываемой системы . . .	11
2.3 Архитектура	12
2.4 Вывод	15
3 Разработка веб интерфейса	16
3.1 Используемые средства разработки	16
3.2 Реализация функционала	17
3.3 Демонстрация реализованного функционала	23
3.4 Вывод	26
Заключение	27
Список использованных источников	29

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

В курсовом проекте используются следующие условные обозначения:

БЗ — база знаний;

OSTIS — Open Semantic Technologies for Intelligent Systems;

SC — Semantic Code;

SCg — Semantic Code graphical;

SCs — Semantic Code string;

SCn — Semantic Code natural.

ВВЕДЕНИЕ

В настоящее время информационные технологии играют важнейшую роль во многих сферах нашей жизни. Одной из таких сфер является область искусственного интеллекта, где разработка и применение интеллектуальных систем становятся все более распространенными. Интеллектуальные системы обладают возможностью анализировать данные, принимать решения и выполнять задачи с высокой точностью и эффективностью, демонстрируя уровень интеллектуальной активности, сравнимый с человеческим.

Однако для полноценного использования и взаимодействия с интеллектуальной системой необходим удобный и интуитивно понятный интерфейс. Веб-интерфейс является одним из наиболее популярных и доступных способов предоставления доступа к функциональности интеллектуальной системы. Он позволяет пользователям взаимодействовать с системой через веб-браузер, не требуя установки дополнительного программного обеспечения.

Тема данного курсового проекта – реализация интерпретатора SC-моделей пользовательских интерфейсов. Одной из подзадач является разработка и реализация веб-интерфейса для интеллектуальной системы. Веб-интерфейс должен предоставлять удобные и интуитивно понятные средства для взаимодействия с системой, позволяя пользователям использовать ее функциональность без необходимости в глубоких знаниях программирования или специализированного обучения. Главной задачей проекта является создание пользовательского интерфейса, который обеспечит эффективное и удобное взаимодействие пользователя с функциональными возможностями интеллектуальной системы.

Для достижения поставленной цели необходимо решить следующие задачи:

- изучить требования и функциональные возможности интеллектуальной системы;
- изучить основы разработки веб-интерфейсов, включая языки программирования, фреймворки и инструменты;
- разработать дизайн и структуру веб-интерфейса, учитывая требования пользователей и особенности интеллектуальной системы;
- реализовать веб-интерфейс, используя выбранные инструменты и технологии;
- протестировать и отладить разработанный веб-интерфейс, убедившись в его корректной работе и соответствии требованиям;
- предоставить документацию по использованию и сопровождению разработанного веб-интерфейса.

В результате успешной реализации данного проекта пользователи смогут взаимодействовать с интеллектуальной системой с помощью удобного

веб-интерфейса, получая доступ к ее функциональности и решая задачи с помощью интеллектуальных алгоритмов и анализа данных.

Кроме того, веб-интерфейс может предоставлять дополнительные возможности, такие как визуализация данных, отображение результатов анализа и обработки, настройка параметров системы и многое другое. Это позволяет пользователям более гибко управлять и взаимодействовать с интеллектуальной системой, расширяя ее функциональность и применение.

В заключение, разработка и реализация веб-интерфейса для интеллектуальной системы является важным шагом в обеспечении эффективного и удобного взаимодействия пользователей с системой. Успешная реализация такого интерфейса позволит раскрыть все возможности интеллектуальной системы и значительно улучшить пользовательский опыт, способствуя применению и принятию решений на основе анализа данных и интеллектуальных алгоритмов.

1 АНАЛИЗ ПОДХОДОВ К СОЗДАНИЮ ВЕБ-ПРИЛОЖЕНИЙ

1.1 Анализ требований к разрабатываемой системе

SC-Web [1] - пользовательский интерфейс, который предоставляет базовые средства просмотра и редактирования базы знаний пользователем, средства для навигации по базе знаний (задания вопросов к базе знаний) и может дополняться новыми компонентами в зависимости от задач, решаемых каждой конкретной OSTIS-системой.



Рисунок 1.1 – Описание требуемых возможностей пользователя

В настоящее время существует реализация системы, которая доступна по адресу `ims.ostis.net`. Однако, в рамках данного курсового проекта планируется разработать новую версию этой системы с использованием инновационных подходов и функциональности, чтобы улучшить ее производительность, надежность и пользовательский опыт.

Новая система должна предоставлять следующие возможности:

- выбор языка интерфейса SC-Web 0.9.0 (русский/английский);
- выбор языка представления выбранного фрагмента базы знаний (естественный язык, SCn, SCg);
- переход на страницу базы знаний IMS (главную страницу);
- поиск любой именованной сущности, имеющейся в базе знаний;
- выбор формата взаимодействия с базой знаний (мануальная навигация посредством нажатия на название соответствующей сущности, перемещение посредством задания вопросов системе);
- навигация по основным разделам базы знаний;
- свертывание списка разделов;
- навигация по истории запросов к базе знаний;
- свертывание истории запросов к базе знаний;
- навигация по базе знаний посредством задания вопросов системе;
- переход к семантической окружности любой сущности, имеющейся в базе знаний;
- навигация по базе знаний посредством вызова меню аргументов;
- закрепление вопросов к системе (аргументы) и задавание вопросов одновременно нескольким аргументам;
- изменение масштаба представляемого фрагмента базы знаний (при выборе SCg в качестве языка представления выбранного фрагмента);
- перемещение любых scg-элементы (узлы, дуги, контуры и др.).

На рисунке 1.1 проиллюстрировано описание требуемых возможностей пользователя в разрабатываемой системе.

1.2 Анализ аналогов разрабатываемой системы

В ходе анализа разрабатываемой системы были рассмотрены следующие аналоги:

Chatsonic [2] - это многофункциональный чат-бот с поддержкой Google-поиска, голосового ввода текста и генерацией картинок по текстовому описанию. Начальное состояние системы изображено на рисунке 1.2.

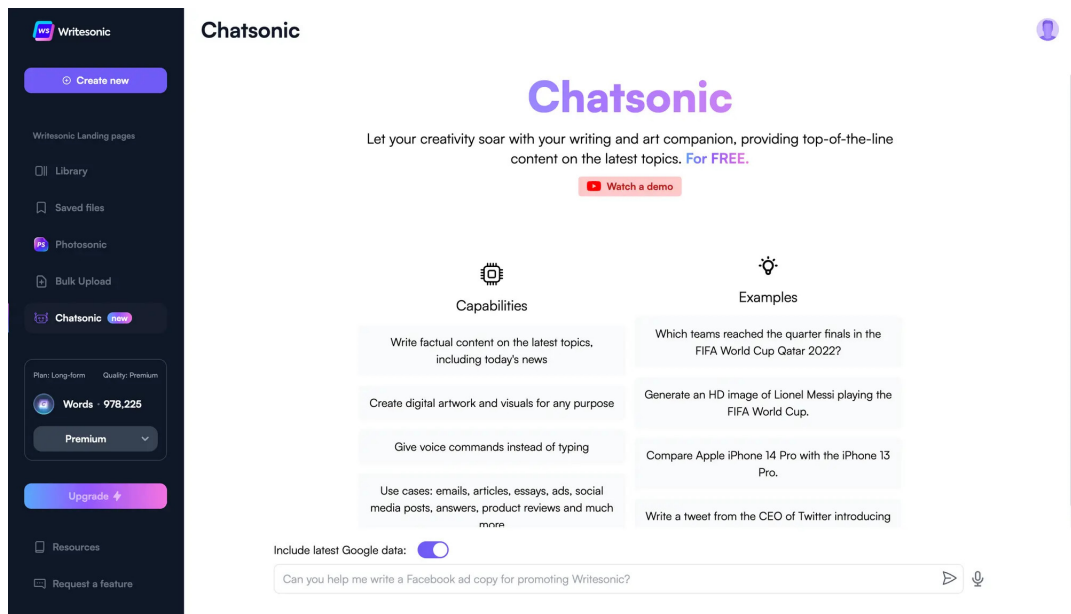


Рисунок 1.2 – Пример начального состояния системы chatsonic.com

Рое.com [3] - это онлайн-платформа, которая предоставляет возможность общения с помощью интеллектуального ассистента в чате. На Рое.com можно выбрать различных интеллектуальных агентов для взаимодействия и отправлять разные типы данных, включая текст и фотографии, в качестве входных данных. Это может быть полезным для получения информации, решения проблем или просто для общения. Начальное состояние системы изображено на рисунке 1.4.

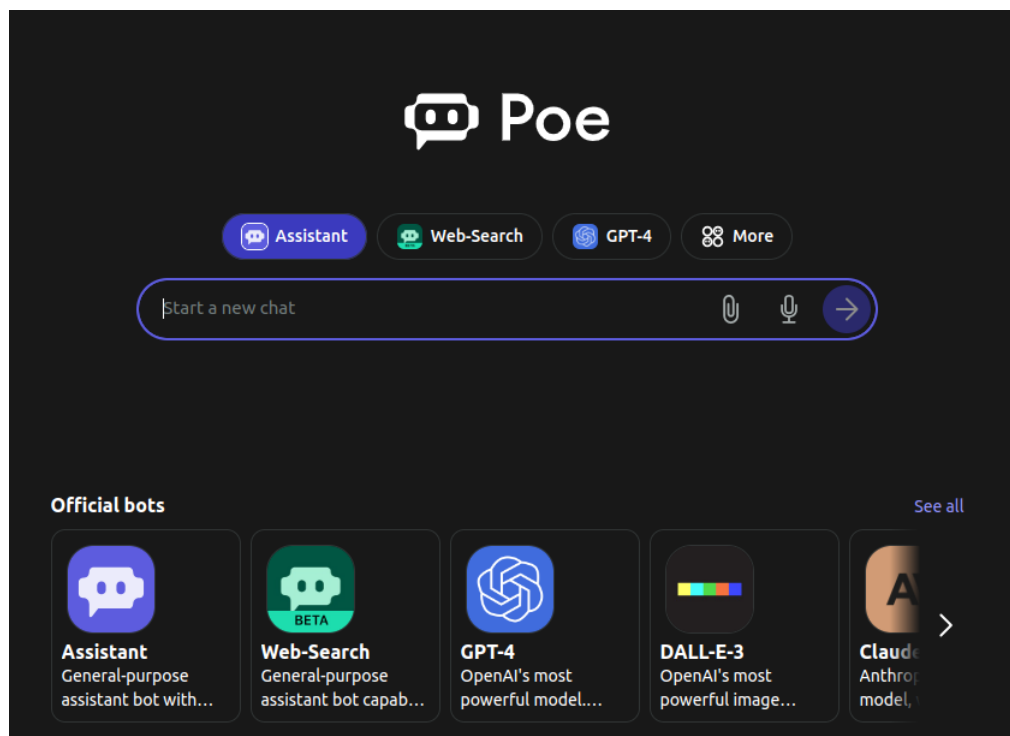


Рисунок 1.3 – Пример начального состояния системы roe.com

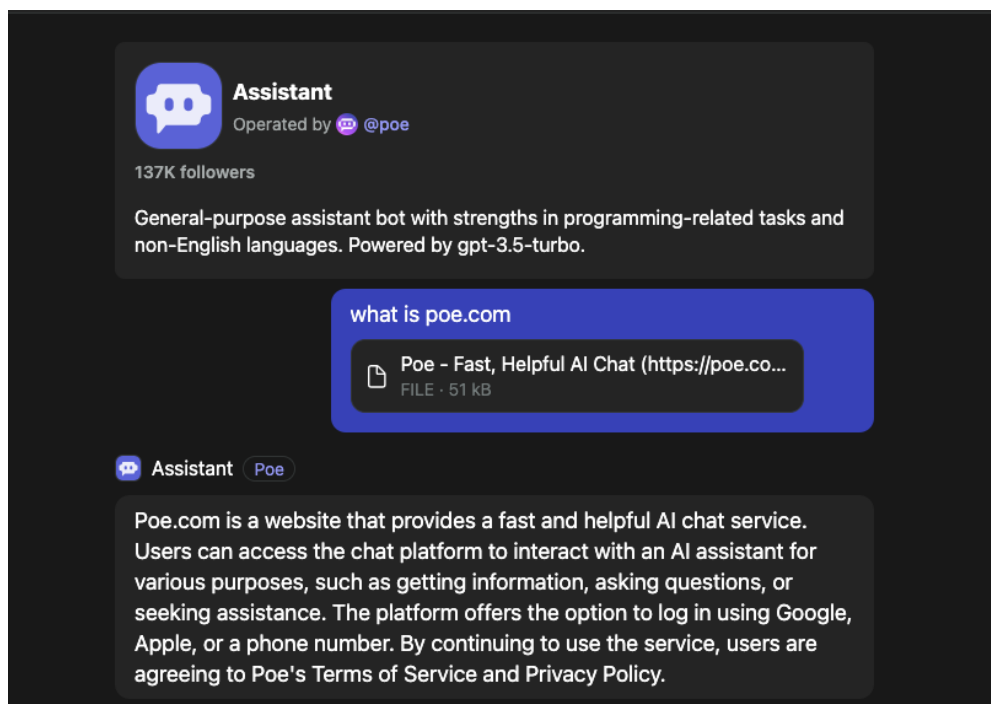


Рисунок 1.4 – Пример компонента диалога системы poe.com

SC-Web представляет собой платформу с несколькими преимуществами по сравнению с аналогичными сервисами. Она позволяет пользователям создавать и визуализировать семантические модели, помогающие более глубоко понять и организовать знания и их взаимосвязи. В отличие от обычных поисковых систем, которые просто предоставляют список релевантных результатов, SC-Web позволяет исследовать и анализировать сложные концепции и связи между ними. Кроме того, можно отправлять шаблонные сообщения для лучшего понимания работы системы. Однако, в отличие от chatsonic.com, SC-Web не позволяет отправлять фотографии в качестве входных данных и в отличие от poe.com, SC-Web не позволяет выбирать между обширным количеством агентов.

1.3 Вывод

В результате проведенного анализа можно сделать следующие выводы:

- 1 Один из ключевых аспектов разработки - провести тщательное исследование аналогов, чтобы понять, какие функции и данные доступны в других языковых интерфейсах. Это позволит определить уникальные возможности и преимущества, которые можно внедрить в SC-Web. На данный момент в SC-Web компонент диалога не предоставляет такого обширного функционала как в рассматриваемых аналогах. Но на основе полученных данных, можно разработать дизайн и функциональность SC-Web, которые будут отвечать потребностям пользователей и предлагать уникальные воз-

возможности по сравнению с аналогами. Однако, SC-Web обладает некоторыми преимуществами, например интуитивный и понятный интерфейс, удобство для длительных переписок и легкость в чтении сообщений.

2 Разработка новой версии SC-Web позволит создать современное, эффективное и масштабируемое веб-приложение с улучшенным пользовательским опытом.

3 Были перечислены требования к разрабатываемой системе, которые позволяют пользователям качественно взаимодействовать с базой знаний.

2 ПРОЕКТИРОВАНИЕ РАЗРАБАТЫВАЕМОЙ СИСТЕМЫ

2.1 Постановка задачи

Разработка новой версии SC-Web должна включать следующие подзадачи:

1 Обновление архитектуры. Проанализировать текущую архитектуру SC-Web и определить области для улучшения и модернизации. Обновить архитектуру, чтобы обеспечить более эффективную обработку данных, масштабируемость и гибкость системы.

2 Расширение функциональности. Исследовать и внедрить новые возможности и функции, которые могут улучшить пользовательский опыт и расширить область применения SC-Web. Примеры включают поддержку дополнительных языков и форматов отображения данных.

3 Улучшение интерфейса пользователя. Продумать и реализовать новый интерфейс пользователя, который будет интуитивно понятным, легким в использовании и эстетически привлекательным. Уделить внимание удобству навигации, визуализации данных и общей пользовательской доступности.

4 Улучшение производительности. Оптимизировать производительность SC-Web, уменьшить время ответа на запросы пользователей и обеспечить более эффективное использование ресурсов системы. Это может включать оптимизацию алгоритмов, кэширование данных и использование параллельных вычислений.

5 Тестирование. Провести обширное тестирование новой версии SC-Web, чтобы обнаружить и исправить возможные ошибки, сбои и неполадки. Убедиться в надежности и стабильности системы перед её выпуском.

2.2 Потенциальные пользователи разрабатываемой системы

Потенциальные пользователи новой версии SC-Web могут включать в себя:

1 Исследователей и ученых. SC-Web может быть полезным инструментом для исследования и анализа сложных знаний и связей между ними. Исследователи из различных областей, таких как биология, физика, социология и другие, могут использовать SC-Web для создания и визуализации семантических моделей и проведения анализа данных.

2 Образовательные учреждения. Университеты, школы и другие образовательные учреждения могут использовать SC-Web в качестве обучающего инструмента для студентов. Он может помочь студентам лучше

понять и организовать сложные концепции и взаимосвязи в различных предметных областях.

3 Разработчиков программного обеспечения. Разработчики могут использовать SC-Web в своих проектах для создания приложений, основанных на семантическом моделировании знаний. SC-Web может предоставить инструменты и API для интеграции с другими системами и использования семантического анализа в различных приложениях.

4 Интернет-пользователей. SC-Web может быть доступен для широкой аудитории интернет-пользователей в целях общего образования, исследования или просто удовольствия. Пользователи могут использовать SC-Web для изучения новых тем, получения информации и взаимодействия с семантическими моделями знаний.

2.3 Архитектура

Архитектура любого проекта играет важную роль в создании масштабируемых, поддерживаемых и эффективных приложений. Хорошо спроектированная архитектура помогает в организации кода, управлении состоянием, коммуникации между компонентами и обеспечении легкой поддержки и расширяемости приложения.

SC-Web – это веб-интерфейс метасистемы OSTIS IMS [4]. Ядром метасистемы является Программная модель SC-памяти (SC-machine) [5], которая одновременно может взаимодействовать как с Реализацией интерпретатора SC-моделей пользовательских интерфейсов (SC-Web), так и с любыми сторонними приложениями по соответствующим сетевым протоколам. С точки зрения общей архитектуры Реализация интерпретатора SC-моделей пользовательских интерфейсов выступает как один из множества возможных внешних компонентов, взаимодействующих с Программной моделью SC-памяти по сети (смотреть рисунок 2.1) [6].

При создании веб-интерфейсов можно использовать различные подходы и архитектурные шаблоны, такие как Flux, Redux, MobX или контекст API, чтобы достичь желаемого уровня организации и управления состоянием. Каждый из этих подходов предлагает свои принципы и инструменты для работы с данными и обновления пользовательского интерфейса.

Хорошая архитектура проекта должна учитывать принципы разделения ответственности, модульности и повторного использования кода. Это способствует легкости тестирования, обновлению и поддержке приложения на протяжении его жизненного цикла. Поэтому в процессе разработки был использован компонентный подход к созданию веб-приложений [7]. Этот подход предполагает разбиение приложения на небольшие независимые компоненты, которые могут быть повторно использованы и легко модифицированы. Каждый компонент отвечает за определенную функциональность

или отображение данных.

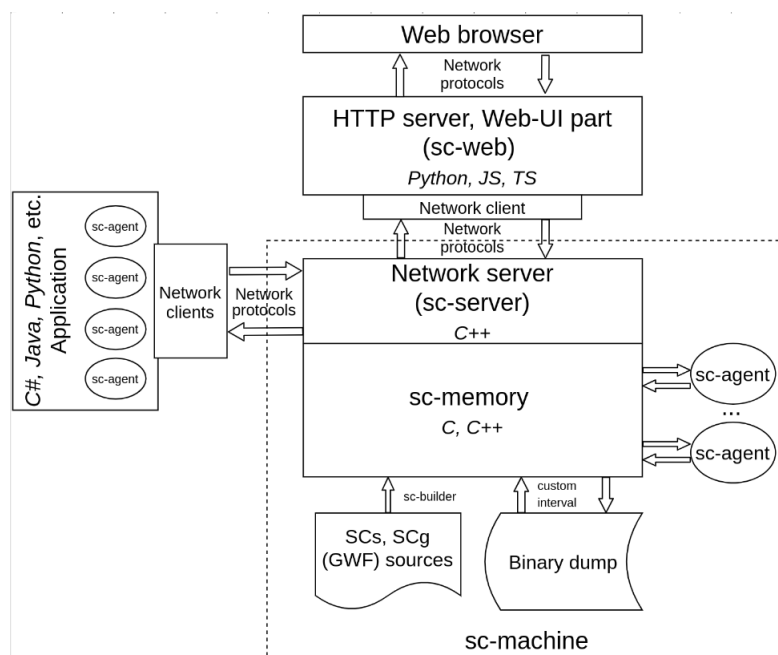


Рисунок 2.1 – Программный вариант реализации платформы интерпретации sc-моделей компьютерных систем

Были спроектированы компоненты, которые отвечают за отображение отдельных элементов пользовательского интерфейса. Основными компонентами пользовательского интерфейса являются компоненты логотипа, боковой панели и поля основного контента. Каждый компонент в свою очередь разбивается на несколько других компонентов.

Также было использовано глобальное хранилище состояния Redux, позволяющее эффективно управлять состоянием приложения и обновлять компоненты при изменении данных.

Помимо управления состоянием, архитектура проекта также включает в себя организацию компонентов, маршрутизацию, обработку событий и взаимодействие с серверной частью. Был использован паттерн контейнер-компонент, который отделяет бизнес-логику от представления и облегчает переиспользование компонентов.

В ходе проектирования были разработаны bpmn-диаграммы для разных бизнес-процессов.

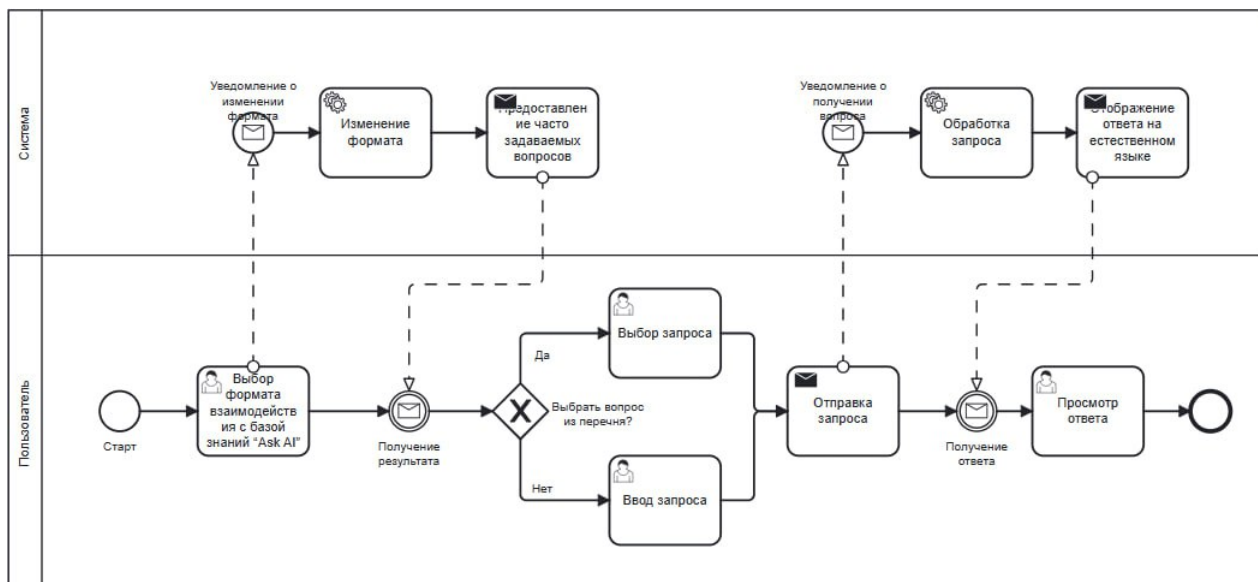


Рисунок 2.2 – Сценарий взаимодействия пользователя с режимом "Ask AI".

На рисунке 2.2 представлена последовательность шагов взаимодействия пользователя с режимом "Ask AI", который осуществляет взаимосвязь между пользователем и системой.

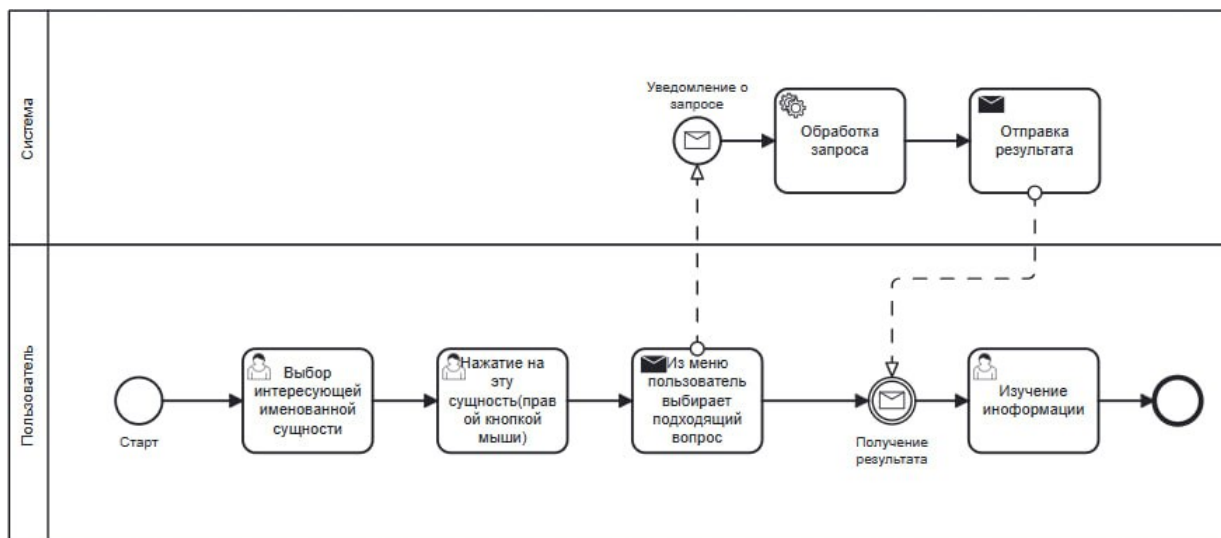


Рисунок 2.3 – Сценарий перемещения пользователя по базе знаний посредством меню.

На рисунке 2.3 представлена последовательность шагов для перемещения по базе знаний посредством вызова меню аргументов (с помощью вопросов).

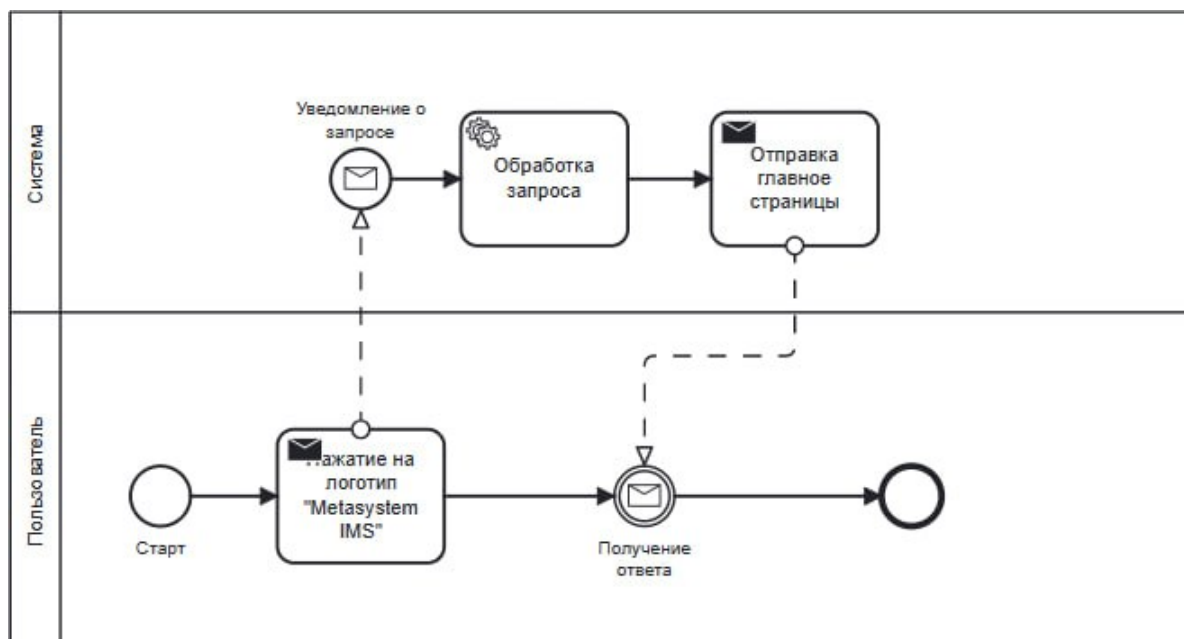


Рисунок 2.4 – Сценарий взаимодействия пользователя с логотипом.

На рисунке 2.4 представлена последовательность шагов взаимодействия пользователя с логотипом, который при нажатии на него отправляет на главную страницу.

2.4 Вывод

В результате проведенного проектирования новой версии SC-Web можно сделать следующие выводы:

1 SC-Web дает возможность интуитивно понятно взаимодействовать с метасистемой OSTIS IMS, которая в свою очередь предоставляет различным группам пользователей полезные инструменты и функциональность, которые облегчают разработку, управление, анализ и использование приложений и систем. Это способствует повышению производительности, эффективности и удовлетворенности пользователей в различных ролях и областях деятельности. Для большей наглядности были предоставлены общие концепты взаимодействия с системой в BPMN диаграммах.

2 В контексте проектирования системы, был выбран компонентный подход к архитектуре. Он должен позволить доработать уже существующий функционал, а также добавить эффективную с точки зрения качества работы и последующей поддержки приложения реализацию нового режима Ask AI.

3 При проектировании новой версии SC-Web были выделены основные компоненты нового режима Ask AI. Это должно увеличить скорость на этапе разработки.

3 РАЗРАБОТКА ВЕБ ИНТЕРФЕЙСА

3.1 Используемые средства разработки

Разрабатываемая система является веб-интерфейсом, поэтому разработка будет вестись с использованием языков программирования JavaScript и TypeScript, также языка разметки HTML и языка стилей CSS с препроцессором SASS.

JavaScript является основным языком программирования для веб-интерфейсов. Он поддерживается всеми современными браузерами и позволяет создавать динамические и интерактивные элементы на веб-страницах. JavaScript обладает широкой функциональностью, включая манипуляцию DOM, обработку событий, выполнение асинхронных запросов и многое другое.

TypeScript является суперсетом JavaScript, который добавляет статическую типизацию и другие возможности разработки, такие как интерфейсы, перечисления, декораторы и т.д. Это позволяет разработчикам обнаруживать ошибки на этапе компиляции, улучшает поддержку кода в IDE и облегчает совместную работу в больших командах. TypeScript является особенно полезным при разработке крупных проектов, где безопасность типов и поддержка инструментов для автоматического рефакторинга имеют большое значение.

HTML (HyperText Markup Language) является стандартным языком разметки для создания структуры и содержимого веб-страниц. Он определяет элементы и их взаимосвязи, такие как заголовки, параграфы, таблицы, изображения и т.д. HTML позволяет разработчикам определить семантическую структуру веб-страницы и является основой для отображения контента.

CSS (Cascading Style Sheets) используется для определения внешнего вида и стиля веб-страниц. С помощью CSS разработчики могут задавать цвета, шрифты, размеры, расположение элементов и другие аспекты визуального представления страницы. CSS позволяет создавать красивый и согласованный дизайн, а также обеспечивает адаптивность и респонсивность интерфейса для различных устройств и экранов.

SASS (Syntactically Awesome Style Sheets) является метаязыком, расширяющим возможности языка CSS. Он предоставляет разработчикам мощные инструменты и функции для более эффективного и организованного написания стилей веб-приложений.

При анализе вышеперечисленных пунктов в качестве основного фреймворка был выбран React [8] в связке с Redux [9]. Так как React и Redux являются популярными инструментами с большим сообществом разработчиков. Это означает, что есть множество готовых решений, библиотек, утилит

и документации, которые помогут в разработке и решении типичных задач. Также React известен своим компонентным подходом, что позволяет создавать масштабируемые и легкоподдерживаемые веб приложения

3.2 Реализация функционала

Для реализации были разработаны компоненты AskElement и AskAnswer.

```
1 import axios from 'axios';
2
3 import { useLocation } from 'react-router-dom';
4 import { useEffect, useReducer, useState } from 'react';
5
6 import { Spinner, useLanguage } from 'ostis-ui-lib';
7 import { SPINER_COLOR } from '@constants';
8 import { AskElement } from './AskElement';
9
10 import styles from './AskAnswer.module.scss';
11 import { AskInput } from '@components/AskInput';
12
13 import { getHintButtonHandler } from 'src/constants/hintButtons';
14
15 interface NavigateState {
16   query?: string;
17   isHintButton: boolean;
18 }
19
20 interface ElementPayload {
21   query: string;
22   answer: string;
23 }
24
25 interface State {
26   history: ElementPayload[];
27 }
28
29 type Action = { type: 'ADD'; payload: ElementPayload } | { type: 'RESET' };
30
31 const historyReducer = (state: State, action: Action): State => {
32   switch (action.type) {
33     case 'ADD':
34       return {
35         ...state,
36         history: [...state.history, action.payload],
37       };
38     case 'RESET':
39       return {
40         ...state,
41         history: [],
42       };
43     default:
44       throw new Error('Unknown action type');
45   }
46 };
47
48 export const AskAnswer = () => {
49   const state = useLocation().state as NavigateState;
50
51   const lang = useLanguage();
52
53   const [historyState, dispatch] = useReducer(historyReducer, { history: [] });
54
55   const [isLoading, setIsLoading] = useState(false);
56
57   const [query, setQuery] = useState<string>();
58
59   const onInputChange = (value: string) => setQuery(value);
60   const onInputSubmit = async () => await fetchAnswerByQuery(query);
61
62   const isDescribeQuery = (query: string): boolean => {
63     const starts =
64       lang === 'ru'
65       ? ['Что это', 'Что это такое', 'Расскажи про']
66       : ['What is', 'Tell me about', 'Describe'];
67     return starts.some((start) => query.startsWith(start));
68   };
69 }
```

Рисунок 3.1 – Реализация компонента AskAnswer

Компонент AskAnswer является частью функционала, предоставляющего возможность пользователям получать ответы от метасистемы OSTIS IMS. Реализация данного компонента изображена на рисунках 3.1 и 3.2.

Компонент AskAnswer использует хук useState для эффективного управления состоянием. Хук useState позволяет сохранять и обновлять значения переменных внутри компонента. В данном случае, используется состояние query, которое представляет текущий вопрос, заданный пользователем.

Кроме того, на странице присутствует компонент AskInput, который предоставляет пользователю возможность вводить свои собственные вопросы. При изменении значения в поле ввода, вызывается функция onChange, которая обновляет состояние query. При отправке формы вызывается функция onSubmit, которая выполняет запрос на получение ответа на вопрос и обновляет состояние истории.

```
70 const fetchAnswerByQuery = async (query: string | undefined) => {
71   console.log(query);
72   if (!query) return;
73
74   setIsLoading(true);
75
76   let answer: string | null = null;
77
78   if (state.isHintButton) {
79     const handler = getHintButtonHandler(lang, state.query!);
80     console.log(handler);
81     answer = await handler();
82   } else if (isDescribeQuery(state.query)) {
83     answer = await getDescriptionOfElement(state.query);
84   }
85
86   if (answer) {
87     dispatch({ type: 'ADD', payload: { query, answer } });
88   } else {
89     dispatch({ type: 'ADD', payload: { query, answer: 'Failed to fetch data' } });
90   }
91
92   setIsLoading(false);
93 };
94
95 useEffect(() => {
96   if (state.query) {
97     fetchAnswerByQuery(state.query);
98   }
99 }, [state]);
100
101 if (!state) return null;
102
103 if (isLoading) return <Spinner className={styles.spinner} appearance={SPINER_COLOR} />;
104
105 return (
106   <div className={styles.pageWrapper}>
107     <div className={styles.history}>
108       {historyState.history.map((entry) => (
109         <AskElement key={entry.query} {...entry} />
110       ))}
111     </div>
112     <AskInput
113       className={styles.inputQueryBar}
114       onChange={onChange}
115       onSubmit={onSubmit}
116     />
117   </div>
118 );
119 };
120
```

Рисунок 3.2 – Реализация компонента AskAnswer

Функция `isDescribeQuery` используется для определения, является ли запрос описанием. В зависимости от типа запроса, выполняется соответствующее действие для получения ответа.

Компонент также использует хук `useEffect` для выполнения запроса при изменении состояния. Если состояние `state` содержит запрос, то вызывается функция `fetchAnswerByQuery` для получения ответа на запрос.

Если компонент находится в состоянии загрузки (`isLoading` равно `true`), отображается компонент `Spinner`. В противном случае, отображается история запросов и компонент `AskInput` для ввода нового запроса.

На рисунке 3.3 изображена реализация компонента `AskElement`.

```
1 import { FC } from 'react';
2 import styles from './AskElement.module.scss';
3
4 import AskAIMessageIcon from '@assets/images/AskAIMessageIcon.svg';
5
6 interface IProps {
7   query: string;
8   answer: string;
9 }
10
11 export const AskElement: FC<IProps> = ({ query, answer }) => {
12   return (
13     <div className={styles.elementWrapper}>
14       <div className={styles.elementQuery}>{query}</div>
15       <div className={styles.elementAnswer}>
16         <div>{<AskAIMessageIcon />}</div>
17         <div>{JSON.stringify(answer)}</div>
18       </div>
19     </div>
20   );
21 };
22
```

Рисунок 3.3 – Реализация компонента `AskElement`

Компонент `AskElement` представляет отдельный элемент истории запросов. Он принимает два свойства - `query` и `answer`, которые отображают текущий вопрос и ответ соответственно.

Внутри компонента имеется обертка `div` с классом `elementWrapper`, которая содержит два дочерних элемента. Первый дочерний элемент с классом `elementQuery` отображает текст текущего вопроса (`query`).

Второй дочерний элемент с классом `elementAnswer` представляет ответ на вопрос. Он состоит из двух частей. Первая часть содержит компонент `AskAIMessageIcon`, который отображает иконку сообщения от искусственного интеллекта. Вторая часть отображает текстовое представление ответа (`answer`), преобразованное в строку с помощью функции `JSON.stringify`.

Для реализации естественно-языкового интерфейса были реализованы следующие компоненты: `NativeLanguagePage` и `NativeLanguage`

```

1 import { routes } from '@constants';
2 import { useTranslate } from 'ostis-ui-lib';
3 import { useMatch } from 'react-router-dom';
4
5 import styles from './NativeLanguagePage.module.scss';
6 import ErrorBoundary from '@components/ErrorBoundary/ErrorBoundary';
7 import { NativeLanguage } from '@components/NativeLanguage/NativeLanguage';
8
9 export const NativeLanguagePage = () => {
10   const translate = useTranslate();
11   const match = useMatch(routes.QUESTION);
12
13   const question = match?.params.question;
14   if (!question) return null;
15
16   return (
17     <div className={styles.wrapper}>
18       <ErrorBoundary
19         title={translate({
20           ru: 'Ошибка загрузки компонента кода естественного языка',
21           en: 'Error loading native language code',
22         })}
23         paragraph={translate({
24           ru: 'Ошибка загрузки компонента кода естественного языка',
25           en: 'Error loading native language code',
26         })}
27         shouldReturn={true}
28       >
29         <NativeLanguage question={Number(question)} />
30       </ErrorBoundary>
31     </div>
32   );
33 };

```

Рисунок 3.4 – Реализация компонента NativeLanguagePage

Компонент NativeLanguagePage служит оберткой над основным компонентом NativeLanguage. Его главной задачей является обеспечение обработки ошибок (с помощью ErrorBoundary) и передача текущего вопроса в компонент NativeLanguage.

```

1 import { IScnNode, Native as NativeLanguageBase, useToast, useTranslate } from 'ostis-ui-lib';
2 import { FC, useCallback, useEffect, useRef, useState } from 'react';
3 import { useNavigate } from 'react-router';
4 import { doCommand } from '@api/requests/command';
5 import { scUtils } from '@api/sc';
6 import { isAxiosError } from '@api/utills';
7 import ArrowIcon from '@assets/images/goBack.svg';
8 import { Button } from '@components/Button';
9 import { Notification } from '@components/Notification';
10 import {
11   DEFAULT_COMMAND_PATH,
12   DEFAULT_COMMAND_SYSTEM_ID,
13   scgUrl,
14   TIMEOUT,
15   TIMEOUT_TOAST_ID,
16 } from '@constants';
17 import { useErrorToast } from '@hooks/useErrorToast';
18 import { useLazyTimeout } from '@hooks/useTimeout';
19
20 import styles from './NativeLanguage.module.scss';
21 import boundaryStyle from '../ErrorBoundary/ErrorBoundary.module.scss';
22 import { getNativeLanguageTree } from '@api/requests/nativeLanguage';
23
24 const timeoutText = {
25   ru: (
26     <>
27       Похоже, ответ на ваш вопрос содержит большое количество связей и его загрузка может
28       выполняться дольше обычного.
29       <span className={styles.waitText}>Пожалуйста, подождите...</span>
30     </>
31   ),
32   en: (
33     <>
34       Your answer contains a large number of links, loading may take more time.
35       <span className={styles.waitText}>Please wait...</span>
36     </>
37   )
38 };
39
40 interface IProps {
41   question: number;
42 }
43
44 export const NativeLanguage: FC<IProps> = ({ question }) => {
45   const [tree, setTree] = useState<IScnNode | null>(null);
46   const [isLoading, setIsLoading] = useState(false);
47
48   const timeoutRequestCountRef = useRef(0);
49
50   const addError = useErrorToast();
51   const { addToast, removeToast } = useToast();
52
53   const translate = useTranslate();
54   const navigate = useNavigate();
55
56   const addTimeoutToast = () => {
57     addToast(
58       <Notification
59         type="warning"
60         title={{ ru: 'Загрузка большого ответа', en: 'Loading complex answer' }}
61         // @ts-ignore
62         text={timeoutText}
63       />,
64       {
65         position: 'bottomRight',
66         id: TIMEOUT_TOAST_ID,
67       },
68     );
69   };
70
71   const { clearTimeout, setTimeout } = useLazyTimeout(addTimeoutToast, TIMEOUT);
72
73   const onRequestStarted = () => {
74     setIsLoading(true);
75     setTree(null);
76   };
77
78   const renderRequestPanel = () => {
79     return null;
80   };
81 }

```

Рисунок 3.5 – Реализация компонента NativeLanguage (часть 1)


```

81
82 const resetTimeoutToast = useCallback(() => {
83   timeoutRequestCountRef.current = 0;
84   removeToast(TIMEOUT_TOAST_ID);
85   clearTimeout();
86 }, [removeToast, clearTimeout]);
87
88 const requestNativeLanguage = useCallback(async () => {
89   setTimeout(() => {
90     const nativeLanguageRes = await getNativeLanguageTree(question, onRequestStarted);
91
92     if (!nativeLanguageRes) {
93       setTree(null);
94       setIsLoading(false);
95       clearTimeout();
96       return;
97     }
98
99     if (!isAxiosError(nativeLanguageRes)) {
100       resetTimeoutToast();
101       setTree(nativeLanguageRes.tree);
102       setIsLoading(false);
103       return;
104     }
105     const isTimeoutError =
106       nativeLanguageRes.response?.status === 404 &&
107       nativeLanguageRes.response.data === 'Timeout waiting for answer translation';
108     if (!isTimeoutError) {
109       resetTimeoutToast();
110       return addError({
111         ru: 'Не удалось получить код естественного языка',
112         en: 'It's failed to get native language code',
113       });
114     }
115     timeoutRequestCountRef.current++;
116     requestNativeLanguage();
117   }, [setTimeout, question, resetTimeoutToast, addError, translate, clearTimeout]);
118
119 const onAskQuestion = async (addr: number) => {
120   const { uiMenuViewFullSemanticNeighborhood } =
121     await scUtils.findKeynodes(DEFAULT_COMMAND_SYSTEM_ID);
122   const cmdRes = await doCommand(uiMenuViewFullSemanticNeighborhood.value, addr);
123
124   if (isAxiosError(cmdRes)) {
125     setIsLoading(false);
126     addError({ ru: 'Не удалось выполнить запрос', en: 'Failed to handle the request' });
127     return null;
128   }
129   return cmdRes.data.question;
130 };
131
132 useEffect(() => {
133   requestNativeLanguage();
134 }, [requestNativeLanguage]);
135
136 useEffect(() => {
137   return resetTimeoutToast;
138 }, [resetTimeoutToast]);
139
140 if (!tree && !isLoading)
141   return (
142     <div className={boundaryStyle.wrap}>
143       <h4>{translate({ ru: 'Понятие было удалено', en: 'Element was deleted' })}</h4>
144       <Button
145         appearance="secondaryAction"
146         className={boundaryStyle.backButton}
147         onClick={() => {
148           navigate(DEFAULT_COMMAND_PATH);
149         }}
150       >
151         <ArrowIcon />
152         {translate({ ru: 'Вернуться на главную страницу', en: 'Go back to main page' })}
153       </Button>
154     </div>
155   );

```

Рисунок 3.6 – Реализация компонента NativeLanguagePage (часть 2)

```

157 return (
158   <NativeLanguageBase
159     tree={tree}
160     scgUrl={scgUrl}
161     isLoading={isLoading}
162     renderRequestPanel={renderRequestPanel}
163     onAskQuestion={onAskQuestion}
164     question={question}
165     className={styles.container}
166   />
167 );
168 };

```

Рисунок 3.7 – Реализация компонента NativeLanguagePage (часть 3)

Компонент NativeLanguage представляет собой комплексную структуру, основной задачей которой является отправка запроса на сервер для получения входных данных (с помощью функции requestNativeLanguage). Для защиты от длительных запросов, в компоненте применяется таймаут, после которого пользователь получает уведомление о невозможности получить данные для представления в естественно-языковом интерфейсе, относящемся к его вопросу.

Для построения дерева, представляющего запрос в естественно-языковом интерфейсе, данный компонент использует компонент NativeLanguageBase, который также был реализован в репозитории ostis-ui-lib. NativeLanguageBase представляет собой сложную реализацию, которая на основе входных данных строит дерево ответа на вопрос в режиме естественно-языкового интерфейса.



Рисунок 3.8 – Структура компонента Native (NativeLanguageBase)

3.3 Демонстрация реализованного функционала

В ходе работы был реализован ряд функциональных возможностей:

Пререход на главную страницу по логотипу

Пользователь при нажатии на логотип платформы по умолчанию переходит из любого другого режима на режим ручной навигации по базе знаний.(рис. 3.9).

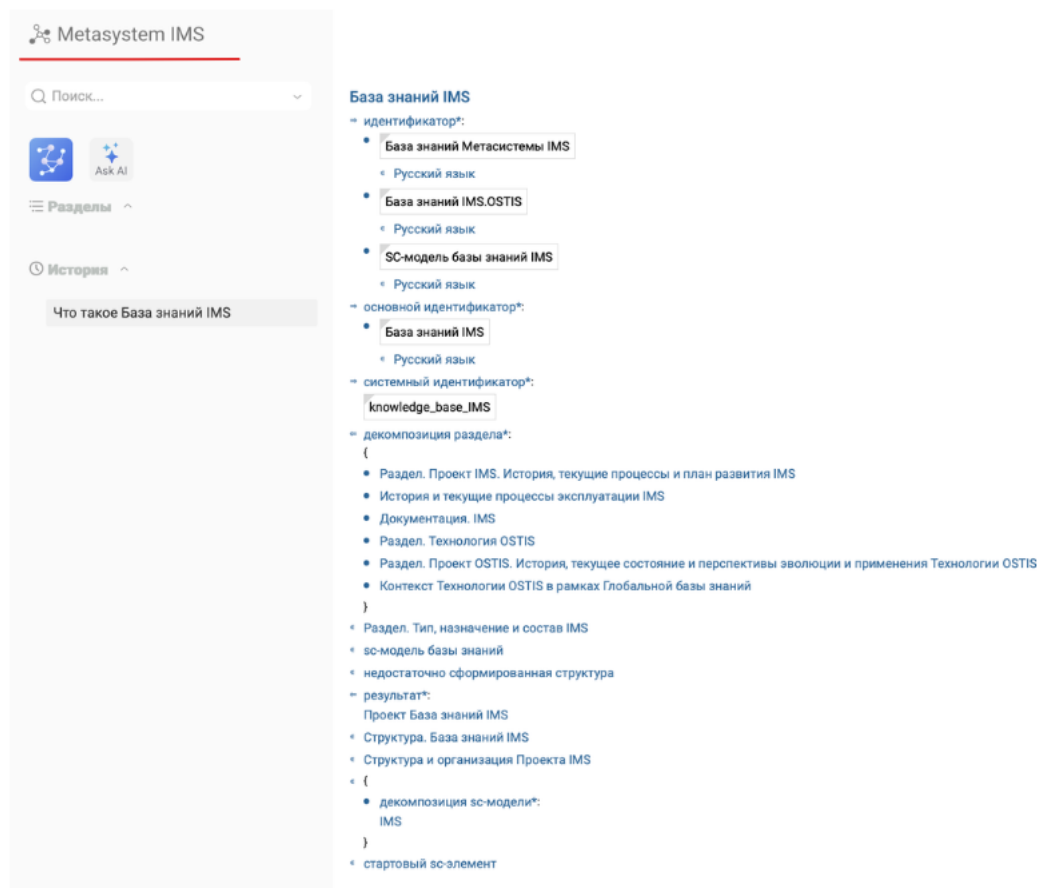


Рисунок 3.9 – страница базы знаний IMS

Формат взаимодействия с базой знаний Ask AI через диалоговое окно

На открывшейся странице пользователь может ввести свой вопрос к системе в строке ввода и задать его системе посредством нажатия на Enter либо нажатия на изображение стрелки (рис. 3.10)

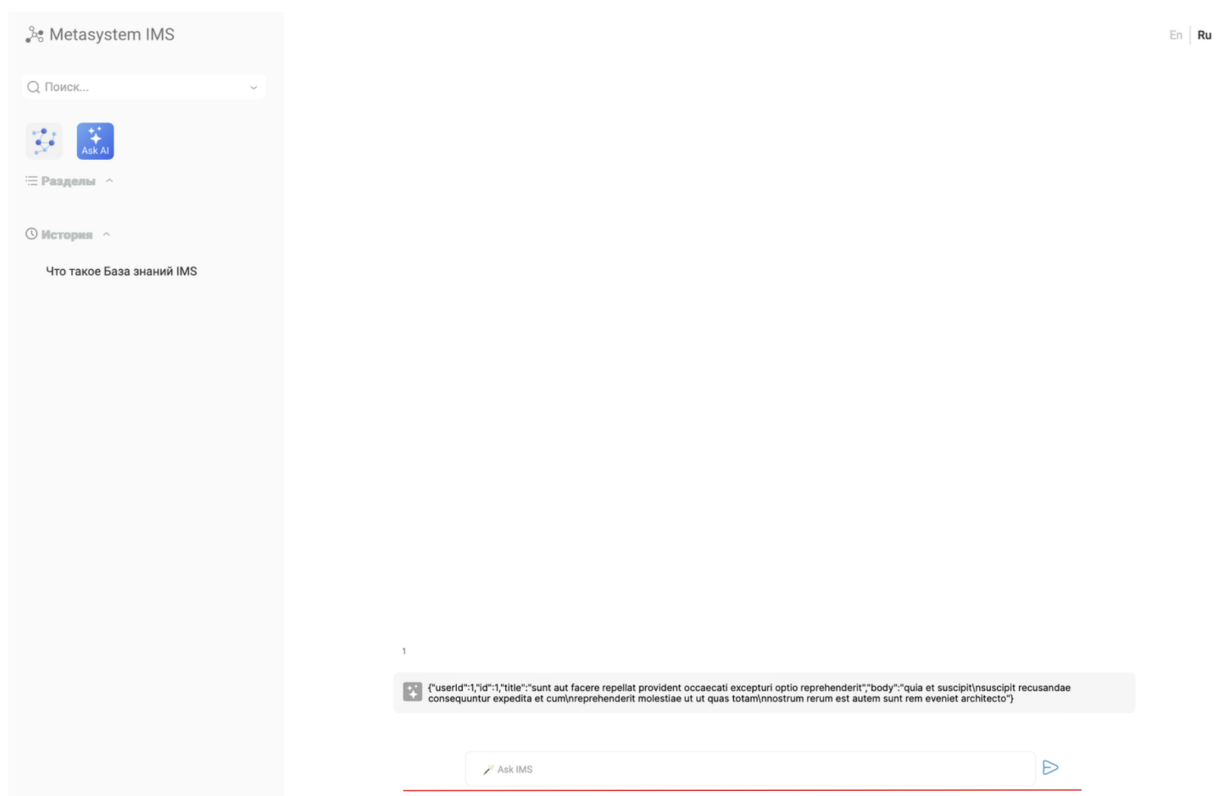


Рисунок 3.10 – Страница Ask AI

Переход в режим **ЕЯ-интерфейса**
 На следующей странице пользователь может видеть текущий запрос в форме естественного языка. (рис. 3.11)



Рисунок 3.11 – ЕЯ-режим

В результате работы над компонентом диалога, пользователи режима AI могут легко и удобно взаимодействовать с метасистемой OSTIS IMS,

задавать вопросы и получать информативные ответы, что в значительной степени улучшает их опыт использования системы.

Результатом работы над режимом естественно-языкового интерфейса (ЕЯ-интерфейса) является возможность пользователям платформы удобно просматривать интересующую их информацию в понятном интерфейсе.

3.4 Вывод

1 Для разработки веб-интерфейса используются языки программирования JavaScript и TypeScript, а также языки разметки HTML и CSS. JavaScript является основным языком для создания динамических и интерактивных элементов на веб-страницах. TypeScript является суперсетом JavaScript, который добавляет статическую типизацию и другие возможности разработки. HTML используется для создания структуры и содержимого веб-страниц, а CSS - для определения их внешнего вида и стиля.

2 В качестве основного фреймворка для разработки выбран React в связке с Redux. React и Redux широко используются в веб-разработке и предоставляют множество готовых решений, библиотек и документации. React также известен своим компонентным подходом, позволяющим создавать масштабируемые и легкоподдерживаемые веб-приложения.

3 Представлен пример демонстрации основного функционала разрабатываемой системы. В демонстрации показывается логотип компании, который может быть использован для идентификации компании и открытия главного экрана системы при его нажатии. Также демонстрируется компонент диалога, который обеспечивает взаимодействие с пользователем, позволяя задавать вопросы и получать ответы от системы. Пользователь может вводить запросы в текстовое поле и получать соответствующие ответы системы.

ЗАКЛЮЧЕНИЕ

Целью курсового проекта была успешная реализация интерпретатора SC-моделей пользовательских интерфейсов, с акцентом на улучшение пользовательского опыта, гибкой архитектуре и расширенных функциональных возможностях.

В начале разработки был проведен анализ требований к системе, определены необходимые возможности, которые должна предоставлять система пользователю. Также был проведен анализ аналогов, выявлены преимущества и недостатки разрабатываемой системы по сравнению с аналогами. Это структурированное и связанное представление информации сыграло ключевую роль в более эффективной разработке новой версии SC-Web.

Затем были выделены основные подзадачи, необходимые для достижения желаемого результата, и определены потенциальные пользователи разрабатываемой системы. Это позволило создать подробные описания и учесть потребности различных категорий пользователей.

В результате была разработана и описана архитектура системы, основанная на компонентном подходе с использованием React и Redux для управления состоянием. Эта спроектированная архитектура обеспечивает гибкую и удобную структуру, которая легко может быть расширена и дополнена в будущем.

Выбор языка программирования TypeScript для реализации проекта интерпретатора SC-моделей пользовательских интерфейсов был обоснованным и имел ряд преимуществ. TypeScript является надмножеством языка JavaScript, добавляющим статическую типизацию и другие возможности, что способствует повышению надежности и поддерживаемости кода. Статическая типизация позволяет выявлять ошибки на этапе разработки и обеспечивает более ясное понимание структуры и взаимодействия компонентов системы.

HTML и CSS были выбраны для разметки и стилей пользовательского интерфейса. HTML является стандартным языком разметки для создания структуры веб-страниц, а CSS позволяет определить внешний вид и стиль элементов на странице. Использование HTML и CSS в проекте обеспечивает гибкость и совместимость с различными браузерами и устройствами.

Для улучшения процесса разработки и поддержки стилей был применен препроцессор SASS (Syntactically Awesome Style Sheets). SASS предоставляет дополнительные возможности, такие как переменные, миксины и вложенность, что упрощает написание и организацию стилей. Он также позволяет использовать функции и операторы, что облегчает создание динамических и адаптивных стилей.

Ожидается, что новая версия SC-Web станет полезным инструментом

для исследования и анализа знаний, а также найдет широкое применение среди образовательных учреждений, разработчиков программного обеспечения и интернет-пользователей.

Кроме того, после завершения разработки и успешной реализации интерпретатора SC-моделей пользовательских интерфейсов, была проведена оценка и анализ полученных результатов. Это позволило выявить достигнутые цели проекта, а также выделить области для дальнейшего улучшения и развития системы.

Курсовой проект по разработке интерпретатора SC-моделей пользовательских интерфейсов привел к созданию улучшенной версии SC-Web, которая предлагает новые возможности и может служить полезным инструментом в различных областях.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Садовский, М. Е. Пользовательский интерфейс экосистемы OSTIS / М. Е. Садовский. — Минск : БГУИР, 2023. — 153 – 158 с.
- [2] Chatsonic.com [электронный ресурс]. — Режим доступа: <https://chatsonic.com/>. — Дата доступа: 16.10.2023.
- [3] Poe.com [электронный ресурс]. — Режим доступа: <https://poe.com/>. — Дата доступа: 14.10.2023.
- [4] OSTIS IMS. — Режим доступа: <https://ims.ostis.net/>. — Дата доступа: 14.10.2023.
- [5] Реализация платформы для web-ориентированных систем, управляемых знаниями [электронный ресурс]. — Режим доступа: https://libeldoc.bsuir.by/bitstream/123456789/3938/1/Koronchik_Realizatsiya.PDF. — Дата доступа: 20.10.2023.
- [6] Документация программного варианта реализации платформы интерпретации sc-моделей компьютерных систем [электронный ресурс]. — Режим доступа: <https://github.com/ostis-ai/ostis-web-platform/blob/develop/docs/main.pdf>. — Дата доступа: 20.10.2023.
- [7] What is Component-Based Architecture? [электронный ресурс]. — Режим доступа: <https://www.mendix.com/blog/what-is-component-based-architecture/>. — Дата доступа: 01.11.2023.
- [8] React Documentation. — Режим доступа: <https://legacy.reactjs.org/docs/getting-started.html>. — Дата доступа: 17.10.2023.
- [9] Redux Documentation. — Режим доступа: <https://redux.js.org/>. — Дата доступа: 17.10.2023.
- [10] Open Semantic Technology for Intelligent Systems [электронный ресурс]. — Режим доступа: <http://conf.ostis.net/>. — Дата доступа: 01.11.2023.